

ANTSDR E300을 위한 PYNQ 3.1.1 포팅 상세 가이드: 단계별 엔지니어링 절차

I. 서론: 포팅 개요 및 아키텍처

A. 포팅 목표

본 보고서의 목적은 ANTSDR E300 특정 하드웨어 보드에서 PYNQ 3.1.1 프레임워크가 완벽하게 동작하는 부팅 가능한 SD 카드 이미지를 생성하는 것입니다. PYNQ 포팅은 하드웨어 플랫폼 정의, 커널 컴파일, 루트 파일 시스템 통합을 아우르는 복잡한 엔지니어링 절차입니다.

본 가이드는 사용자가 개발 환경 설정부터 최종 이미지 빌드 및 부팅 검증까지의 전 과정을 직접 수행할 수 있도록 "따라하기 수준(follow-along level)"의 상세한 기술 매뉴얼로 작성되었습니다. 모든 단계별 명령어, 설정 파일 수정 사항, 그리고 예상되는 결과를 명확하게 제시하여 절차적 모호성을 제거하는 것을 목표로 합니다.

B. ANTSDR E300 하드웨어 아키텍처 (요약)

포팅 프로세스는 대상 하드웨어의 고유한 구성에 맞춰져야 합니다. ANTSDR E300 보드의 아키텍처는 다음과 같은 핵심 구성 요소를 기반으로 합니다.

- **핵심 SoC:** Xilinx Zynq-7000 SoC. 본 가이드에서는 XC7Z030-1SBG485C 부품을 기준으로 절차를 진행합니다.
- **주요 페리페럴 (Processing System, PS):**
 - DDR3 메모리: 2GB Micron DDR3 SDRAM.
 - QSPI 플래시 메모리.
 - 이더넷 PHY: Realtek RTL8211CL 기가비트 이더넷 트랜시버.
 - SD 카드 인터페이스 (부팅 및 루트 파일 시스템용).

- UART (콘솔 출력 및 제어용).
- 핵심 기능 (**Programmable Logic, PL**):
 - RF 프런트엔드: Analog Devices AD9361 RF Transceiver. 이는 ANTSDR E300의 핵심 SDR(Software Defined Radio) 기능입니다.

C. PYNQ 3.1.1 아키텍처

PYNQ (Python Productivity for Zynq)는 Zynq 플랫폼의 생산성을 높이기 위해 설계된 오픈 소스 프레임워크입니다. PYNQ 3.1.1 스택은 크게 세 부분으로 구성됩니다.

1. 운영체제(**OS**): Petalinux 또는 Yocto 프로젝트를 기반으로 빌드된 임베디드 Linux. PYNQ 3.1.1은 Ubuntu 22.04 LTS 기반의 루트 파일 시스템을 사용합니다.
2. **Python** 환경: Jupyter Notebook을 포함한 Python 런타임 및 라이브러리.
3. **PYNQ** 라이브러리: pynq Python 모듈. 이 라이브러리는 PL(FPGA)에 로드된 하드웨어 디자인(오버레이, Overlay)을 Python 객체처럼 관리하고 제어할 수 있도록 추상화 계층을 제공합니다.

D. 포팅 프로세스 4단계 요약

본 가이드는 ANTSDR E300 하드웨어에 PYNQ 3.1.1을 포팅하기 위한 전체 과정을 다음과 같은 4가지 핵심 엔지니어링 단계로 나누어 순차적으로 진행합니다.

1. **1단계 (Vivado):** ANTSDR E300의 하드웨어 구성(PS 페리페럴, PL IP)을 정의하고 Xilinx Support Archive (XSA) 파일을 생성합니다.
2. **2단계 (PYNQ sdbuild):** PYNQ 빌드 시스템인 sdbuild 내에 ANTSDR E300 보드를 위한 새로운 보드 패키지(BSP) 명세(specification)를 생성합니다.
3. **3단계 (Petalinux/Kernel):** ANTSDR E300의 특정 하드웨어(예: 이더넷 PHY, SDR 트랜시버)를 구동하기 위한 커널 드라이버를 활성화하고, 디바이스 트리(Device Tree)를 커스터마이징합니다.
4. **4단계 (sdbuild):** PYNQ sdbuild를 실행하여 1, 3단계의 결과물을 PYNQ 3.1.1 루트 파일 시스템과 병합하여 최종 부팅 가능한 SD 카드 이미지(.img)를 생성합니다.

II. 개발 환경 구축 및 사전 요구사항

포팅 프로세스를 시작하기 전에, 모든 도구 체인과 소스 코드가 정확한 버전을 유지하도록

호스트 개발 환경을 구축해야 합니다.

A. 호스트 시스템 요구사항

- 권장 **OS**: PYNQ 3.1.1 및 Xilinx 2023.1 도구와의 호환성을 위해 Ubuntu 20.04 LTS 또는 22.04 LTS 사용을 강력히 권장합니다.
- 필수 패키지 설치: **sdbuild** 및 **Petalinux** 빌드에 필요한 종속성 패키지를 설치해야 합니다.

Bash

```
# PYNQ sdbuild 및 Petalinux 2023.1 구동을 위한 필수 패키지
```

```
sudo apt-get update
```

```
sudo apt-get install -y build-essential gcc-multilib g++-multilib \
libssl-dev autoconf automake libtool-bin bison flex zlib1g-dev \
libncurses5-dev libncursesw5-dev device-tree-compiler \
tofrodo python3-pip python3-venv git gawk wget cpio diffstat \
chrpath socat xterm rsync texinfo iputils-ping \
libegl1-mesa libwayland-client0 libwayland-cursor0 \
pax dh-autoreconf
```

B. Xilinx 도구 설치 (버전 동기화)

PYNQ 3.1.1 프레임워크는 Xilinx 2023.1 버전의 도구 체인과 완벽한 호환성을 갖도록 설계되었습니다. 버전 불일치는 빌드 실패의 주된 원인이므로, 반드시 지정된 버전을 설치해야 합니다.

- 필수 설치:
 - Vivado ML Edition 2023.1
 - Vitis Unified Software Platform 2023.1
 - Petalinux 2023.1
- 환경 변수 설정: Xilinx 도구를 사용하기 전에, 터미널 세션에서 관련 환경 변수를 설정(source)해야 합니다.

Bash

```
# Vivado/Vitis 2023.1 환경 설정
```

```
source /opt/Xilinx/Vivado/2023.1/settings64.sh
```

```
# Petalinux 2023.1 환경 설정
```

```
source /opt/Xilinx/Petalinux/2023.1/settings.sh
```

(주: 설치 경로는 사용자 환경에 따라 다를 수 있습니다.)

C. PYNQ 3.1.1 소스 코드 복제

PYNQ 빌드 시스템(sdbuild)을 포함한 PYNQ 3.1.1 태그의 공식 소스 코드를 복제합니다.

Bash

```
# v3.1.1 태그를 지정하여 복제
```

```
git clone --recursive --branch v3.1.1 https://github.com/Xilinx/PYNQ.git
```

--recursive 옵션은 PYNQ가 의존하는 sdbuild와 같은 하위 Git 모듈을 함께 복제하기 위해 필수적입니다.

D. (표) 필수 소프트웨어 및 버전

PYNQ 포팅 실패의 가장 큰 원인은 도구 체인 버전 불일치입니다. 다음 표는 ANTSDR E300 포팅에 필수적인 소프트웨어와 버전 요구사항을 요약한 것입니다.

소프트웨어	필수 버전	비고
호스트 OS	Ubuntu 20.04/22.04 LTS	
PYNQ	v3.1.1	git clone --branch v3.1.1
Xilinx Vivado	2023.1	
Xilinx Petalinux	2023.1	

PYNQ 3.1.1과 Xilinx 2023.1 도구 간의 버전 종속성은 단순한 권장 사항이 아닙니다. 이는 하드웨어 정의(XSA)부터 커널(Petalinux) 및 루트 파일 시스템(PYNQ)까지 이어지는 전체 빌드

체인의 *무결성(integrity)*을 보장하는 핵심 요구사항입니다.

이 관계는 다음과 같은 논리적 연쇄를 따릅니다:

1. Vivado 2023.1은 특정 버전의 IP 카탈로그와 하드웨어 메타데이터 스키마를 사용하여 XSA 파일을 생성합니다.
2. Petalinux 2023.1은 이 특정 2023.1 XSA 스키마를 파싱하여 디바이스 트리의 기본 골격과 부트로더(U-Boot) 설정을 생성하도록 설계되었습니다.
3. PYNQ 3.1.1의 sdbuild 스크립트는 이 Petalinux 2023.1의 출력(예: Linux 커널 버전 6.1)을 정확히 기대합니다.
4. PYNQ의 Python 라이브러리(예: pynq.Overlay 모듈)는 이 특정 커널 버전(6.1)에서 제공하는 UIO(Userspace I/O) 또는 DMA 드라이버 인터페이스와 호환되도록 빌드됩니다.

만약 사용자가 임의로 Vivado 2022.2와 같은 다른 버전을 사용한다면, XSA 메타데이터 스키마가 일치하지 않아 Petalinux가 XSA를 파싱하는 단계에서부터 오류(예: [Common 17-55] 오류)가 발생할 수 있습니다. 설령 빌드가 성공하더라도, 런타임에 PYNQ Python 라이브러리와 커널 드라이버 간의 API 불일치로 인해 오버레이 로드가 실패하는 등 예측 불가능한 오류가 발생하게 됩니다. 따라서 버전 일치는 포팅의 첫 번째이자 가장 중요한 단계입니다.

III. 1단계: ANTSDR E300 하드웨어 플랫폼 정의 (Vivado 2023.1)

첫 번째 단계는 Vivado 2023.1을 사용하여 ANTSDR E300 보드의 하드웨어 구성을 정의하는 것입니다. 이 결과물인 XSA 파일은 후속되는 모든 소프트웨어 빌드(Petalinux, PYNQ)의 기반이 됩니다.

A. Vivado 프로젝트 생성

1. Vivado 2023.1을 실행합니다.
2. 새 프로젝트를 생성합니다 (예: ANTSDR_E300).
3. "RTL Project"를 선택하고, "Do not specify sources at this time"을 체크합니다.
4. "Default Part" 선택 단계에서 ANTSDR E300의 Zynq 부품을 선택합니다.
 - Part: XC7Z030-1SBG485C

B. 블록 디자인 생성 및 Zynq-7000 PS(Processing System) 설정

1. "Flow Navigator"에서 "Create Block Design"을 선택합니다.
2. "Add IP"를 클릭하여 "Zynq 7 Processing System"을 추가합니다.
3. "Run Block Automation"을 실행하여 Zynq PS의 기본 설정을 적용합니다 (DDR, FIXED_IO).

C. Zynq PS 상세 설정 (ANTSDR E300 맞춤화)

생성된 Zynq PS 블록을 더블 클릭하여 ANTSDR E300 보드 스키매틱에 맞게 상세 설정을 진행합니다. 이는 포팅에서 매우 중요한 과정입니다.

1. **DDR3** 메모리 설정:
 - "DDR Configuration" 탭으로 이동합니다.
 - ANTSDR E300은 2GB Micron DDR3를 사용합니다. "DDR Controller Configuration"에서 "Memory Part"를 MT41K512M16 (Micron 2Gb) 또는 유사한 프리셋으로 선택합니다.
 - 주의: 프리셋을 사용하더라도, 보드 스키매틱을 참조하여 정확한 CAS Latency, T_{RC}, T_{RAS} 등의 타이밍 파라미터를 수동으로 검증하고 입력해야 합니다.
2. **MIO** 및 페리페럴 설정:
 - "MIO Configuration" 탭으로 이동합니다.
 - **Ethernet (GEMO):** "Peripheral I/O Pins"에서 Ethernet 0를 활성화합니다.
 - ANTSDR E300은 Realtek RTL8211CL PHY를 사용합니다. 보드 스키매틱에 따라 MDIO 인터페이스 핀(MDC, MDIO)이 MIO 50, 51에 연결되어 있습니다.
 - MDIO 핀을 MIO 50으로, MDC 핀을 MIO 51로 수동 설정합니다. (EMIO가 아닌 MIO를 사용하도록 명시해야 합니다).
 - **UART 1:** UART 1을 활성화하고 핀을 MIO 48 (TX), MIO 49 (RX)로 설정합니다 (부팅 로그 및 셸 접속용).
 - **SDIO 0 (SD Card):** SD 0를 활성화합니다 (MIO 40-45). 부팅에 필수적입니다.
 - **QSPI Flash:** Quad SPI Flash를 활성화합니다.

Vivado에서 이더넷(GEMO)을 MIO 50/51로 설정하는 것은 단순한 핀 할당이 아닙니다. 이 작업은 생성될 XSA 파일 내부에 "이 하드웨어는 GEMO용 MDIO 버스를 MIO 50/51 핀을 통해 PS에서 직접 제어한다"는 하드웨어 메타데이터를 저장합니다. 이 정보가 없으면, Petalinux는 디바이스 트리에서 &gem0 노드의 MDIO 버스를 올바르게 정의하지 못합니다.

만약 이 단계를 누락하고 기본값(예: EMIO)으로 두면, XSA에 MIO 50/51 정보가 포함되지 않습니다. Petalinux는 이 XSA를 기반으로 디바이스 트리를 생성하며, &gem0 노드에 MDIO 버스 정의가 빠지게 됩니다. 이 경우, 사용자가 나중에 system-user.dtsi에서 PHY를 정의하더라도, 해당 PHY가 연결될 MDIO 버스 자체가 커널에 등록되지 않습니다. 결과적으로 부팅 시 "PHY driver not found"가 아니라, MDIO 버스 자체를 찾지 못해 이더넷 인터페이스가 아예 활성화되지 않는(link down) 더 심각한 오류가 발생합니다. 따라서 Vivado 단계는 모든

후속 소프트웨어 단계의 전제 조건입니다.

E. (SDR) AD9361 RF Transceiver IP 추가

ANTSDR E300의 핵심 기능은 AD9361 RF Transceiver입니다. 이 칩은 PS 페리페럴이 아니라 PL(FPGA 로직)에 연결됩니다.

1. Analog Devices의 공식 HDL/IP 라이브러리를 Vivado 프로젝트에 IP Repository로 추가해야 합니다.
2. 블록 디자인에서 "Add IP"를 클릭하여 "AXI AD9361" IP를 추가합니다.
3. "Run Connection Automation"을 실행하여 Zynq PS의 AXI 인터커넥트(데이터 및 제어용), FCLK(클럭), 리셋 신호에 자동으로 연결합니다.
4. 중요: AD9361 IP에서 발생하는 인터럽트(Interrupt) 출력 핀을 Zynq PS의 IRQ_F2P[0:0] (FPGA-to-PS Interrupt) 핀에 수동으로 연결해야 합니다. 이 연결은 나중에 Linux 커널의 IIO 드라이버가 AD9361의 이벤트를 비동기적으로 처리하는 데 필수적입니다.

F. XSA (Xilinx Support Archive) 생성

모든 하드웨어 설정이 완료되면, Petalinux와 PYNQ가 사용할 하드웨어 정의 파일(XSA)을 생성합니다.

1. 블록 디자인 다이어그램에서 "Validate Design" (F6)을 실행하여 오류가 없는지 확인합니다.
2. "Sources" 탭에서 블록 디자인 파일(.bd)을 마우스 오른쪽 버튼으로 클릭하고 "Create HDL Wrapper"를 선택합니다.
3. "Flow Navigator"에서 "Generate Bitstream"을 실행합니다. (이 과정은 PYNQ의 기본 오버레이(bitstream)가 되며, 시간이 다소 소요됩니다.)
4. Bitstream 생성이 성공적으로 완료되면, File -> Export -> Export Hardware...를 선택합니다.
5. 플랫폼 유형으로 "Fixed"를 선택합니다.
6. "Include Bitstream" 옵션을 반드시 체크합니다.
7. 파일 이름을 ANTSDR_E300.xsa로 지정하고 저장합니다.

IV. 2단계: PYNQ 보드 패키지 생성 (sdbuild)

이제 PYNQ sdbuild 빌드 시스템에 ANTSDR E300 보드를 "알려주는" 보드 패키지 파일을 생성합니다.

A. sdbuild 디렉토리 구조 생성

II-C 단계에서 복제한 PYNQ 소스 코드 디렉토리로 이동합니다. sdbuild는 PYNQ/sdbuild에 위치하지만, 보드 정의는 PYNQ/boards 디렉토리에 위치합니다.

Bash

```
# PYNQ 소스 코드 최상위 디렉토리로 가정
cd PYNQ
mkdir -p boards/ANTSDR_E300/
```

B. ANTSDR_E300.spec 파일 작성

sdbuild의 빌드 명세 파일인 .spec 파일을 생성합니다. 이 파일은 sdbuild에게 ANTSDR E300 보드에 대해 필요한 모든 정보(아키텍처, XSA 경로, BSP)를 알려줍니다.

boards/ANTSDR_E300/ANTSDR_E300.spec 파일을 생성하고 다음 내용을 작성합니다:

Ini, TOML

```
[spec]
# ANTSDR E300 (Zynq-7000)의 아키텍처
ARCH_ANTSDR_E300 = armv7a

# 1단계(Vivado)에서 생성한 XSA 파일의 *절대 경로* 지정
# 경로는 사용자 환경에 맞게 수정해야 함
XSA_ANTSDR_E300 = /path/to/vivado_project/ANTSDR_E300.xsa
```



```
# PYNQ의 공통 Zynq BSP를 상속받음
# 이 BSP는 PYNQ 라이브러리, Jupyter 등 공통 패키지를 정의함
BSP_ANTSDR_E300 = boards/Zynq/zynq.bsp
```

```
# sdbuild가 사용할 Petalinux 구성 오버라이드 디렉토리
# 3단계에서 수정할 디바이스 트리, 커널 설정 파일이 위치할 경로
PETALINUX_CONFIG_ANTSDR_E300 = boards/ANTSDR_E300/petalinux_config/
```

ANTSDR_E300.spec 파일은 PYNQ sdbuild의 핵심 동작을 정의합니다. BSP_ANTSDR_E300 = boards/Zynq/zynq.bsp 라인은 ANTSDR E300이 Zynq-7000 보드용 공통 PYNQ 설정 (예: Jupyter 패키지, PYNQ Python 라이브러리, 기본 Ubuntu 패키지 목록)을 상속받는다라는 것을 의미합니다.

sdbuild는 이 .spec 파일을 읽고 다음과 같은 3단계 오버레이(overlay) 방식으로 빌드를 수행합니다:

1. 기반(**Base**): zynq.bsp의 공통 Petalinux 설정을 로드합니다.
2. 하드웨어 오버레이: XSA_ANTSDR_E300에 지정된 ANTSDR_E300.xsa 파일을 사용하여 이 설정을 덮어쓰기 합니다. 이 단계에서 Vivado의 하드웨어 정의가 Petalinux에 적용됩니다.
3. 보드 특화 오버레이: PETALINUX_CONFIG_ANTSDR_E300 디렉토리(우리가 3단계에서 채울)의 설정으로 다시 한번 덮어쓰기 합니다. 이 단계에서 커스텀 디바이스 트리와 커널 드라이버 설정이 적용됩니다.

이 3단계 오버레이 방식은 PYNQ가 수많은 Zynq 보드를 효율적으로 지원하는 핵심 메커니즘입니다. 이 구조를 이해해야 3단계에서 system-user.dtsi 파일을 어디에 위치시켜야 하는지, 그리고 왜 그렇게 해야 하는지 명확히 알 수 있습니다.

D. 3단계를 위한 디렉토리 준비

sdbuild가 3단계의 커스텀 설정을 찾을 수 있도록 .spec 파일에 명시한 petalinux_config 디렉토리나 그 하위 표준 경로를 미리 생성합니다.

Bash

```
# ANTSDR E300 보드 디렉토리 내에 petalinux_config 디렉토리 생성
cd boards/ANTSDR_E300
```

```
# 디바이스 트리 파일(.dtsi)이 위치할 경로
```

```
mkdir -p petalinux_config/project-spec/meta-user/recipes-bsp/device-tree/files/
```

커널 설정 조각(.cfg)이 위치할 경로

```
mkdir -p petalinux_config/project-spec/meta-user/recipes-kernel/linux/files/
```

이 경로는 Petalinux/Yocto 빌드 시스템이 사용자 정의 레이어(meta-user)에서 설정 파일을 찾는 표준 경로입니다.

V. 3단계: Petalinux 프로젝트 구성 및 커널 커스터마이징

이 단계는 PYNQ sdbuild가 4단계에서 Petalinux를 빌드할 때 사용할 설정 파일을 미리 준비하는 과정입니다. 실제 Petalinux 명령어를 실행하는 것이 아니라, PYNQ/boards/ANTSDR_E300/petalinux_config/ 디렉토리 내에 필요한 파일들을 작성하는 작업입니다.

A. 커널 드라이버 활성화

ANTSDR E300의 특정 하드웨어(이더넷 PHY, SDR 트랜시버)를 사용하려면, Linux 커널 빌드 시 해당 드라이버 모듈이 활성화(컴파일)되어야 합니다.

IV-D에서 생성한 경로에 bsp.cfg 파일을 생성합니다.

파일 위치:

```
PYNQ/boards/ANTSDR_E300/petalinux_config/project-spec/meta-user/recipes-kernel/linux/files/bsp.cfg
```

파일 내용:

코드 스니펫

```
# ANTSDR E300 커널 설정 조각 (bsp.cfg)
```

```
# 1. 이더넷 PHY (Realtek RTL8211CL) 드라이버 활성화
CONFIG_REALTEK_PHY=y
```

2. AD9361 (SDR) IIO 드라이버 활성화

```
CONFIG_AD9361=y
CONFIG_IIO=y
CONFIG_IIO_BUFFER=y
CONFIG_IIO_KFIFO_BUF=y
CONFIG_IIO_TRIGGER=y
CONFIG_IIO_SUBSYSTEM=y
```

3. 기타 ADI IIO 관련 필수 드라이버

(AD9361은 SPI를 통해 제어되므로)

```
CONFIG_SPI_MASTER=y
CONFIG_SPI_XILINX=y
```

(기타 필요한 IIO 드라이버들...)

이 bsp.cfg 파일은 Petalinux가 kernel_menuconfig를 실행할 때 기본 defconfig에 추가로 적용되는 "설정 조각(fragment)"입니다.

sdbuild가 Petalinux 빌드를 시작하면, XSA에 기반한 기본 커널 설정을 생성합니다. 이 기본 설정에는 CONFIG_REALTEK_PHY나 CONFIG_AD9361과 같은 특수 드라이버들이 비활성화(Not Set)되어 있을 가능성이 높습니다. Petalinux 빌드 시스템은 meta-user 레이어에서 이 bsp.cfg 파일을 찾아, 해당 설정들을 강제로 y(활성화)로 변경하여 커널에 포함시킵니다.

만약 이 파일이 누락되면, 3-A단계에서 디바이스 트리(VI 단계)를 아무리 잘 작성해도, 커널 이미지(.ko 또는 내장)에 해당 드라이버 코드가 아예 존재하지 않게 됩니다. 결과적으로 부팅 시 "driver not found" 오류가 발생합니다.

VI. 3-A단계: 디바이스 트리(Device Tree) 심층 수정

이것이 PYNQ 포팅에서 가장 중요하고 실패하기 쉬운 단계입니다. 디바이스 트리는 Linux 커널에게 하드웨어의 구성(어떤 칩이, 어떤 버스의, 어떤 주소에, 어떤 인터럽트를 사용하는지)을 알려주는 "지도"입니다.

V 단계에서 준비한 meta-user 경로에 system-user.dtsi (Device Tree Source Include) 파일을 생성합니다.

파일 위치:

```
PYNQ/boards/ANTSDR_E300/petalinux_config/project-spec/meta-user/recipes-bsp/device-tree/files/system-user.dtsi
```

파일 내용:

DTS

```
/include/ "system-conf.dtsi"
/{
/*
* ANTSDR E300 보드에 대한 사용자 정의 디바이스 트리
*/
};

/*
* 1. 이더넷 PHY (RTL8211CL) 설정
* Vivado에서 GEM0 (Ethernet 0)가 MIO 50/51로 설정된 것을 기반으로 함
*/
&gem0 {
    status = "okay";
    phy-mode = "rgmii-id"; /* RGMII with Internal Delay (RTL8211CL 지원 모드) */
    phy-handle = <&eth_phy0>;

    mdio {
        /* 이 노드는 mdio 버스를 정의하며, 그 하위에 PHY를 정의함 */
        eth_phy0: phy@1 { /* PHY가 MDIO 버스의 주소 1번에 위치함 (스키매틱 확인 필요) */
            reg = ;
            compatible = "realtek,rtl8211cl"; /* 커널 드라이버 매칭용 키 */
            device_type = "ethernet-phy";
        };
    };
};

/*
* 2. AD9361 (SDR) 설정 (PL에 연결된 AXI IP)
* Vivado에서 AXI AD9361 IP의 주소에 맞게 수정 필요
*/
/* 예: AXI 주소가 0x43C00000이고, 주소 범위가 64K(0x10000)일 경우 */
axi_ad9361: ad9361@43C00000 {
    compatible = "adi,ad9361-phy"; /* IIO 드라이버 매칭용 키 */
    reg = <0x43C00000 0x10000>;

/*
* Vivado에서 PS의 IRQ_F2P에 연결한 인터럽트 번호
* Zynq GIC에서 F2P IRQ는 61번부터 시작 (61+0 = 61)
*/
```

```

* GIC_SPI (32)를 빼면 61-32 = 29.
* <0 29 4> (GIC_SPI, IRQ 29, Level-high) - 이 값은 확인이 필요함
*/
interrupts = <0 29 4>;
interrupt-parent = <&gic>;

/*
* AD9361이 사용하는 SPI 버스에 대한 참조 (phandle)
* (만약 별도의 SPI 버스에 연결되어 있다면)
*... spi-master = <&spi0>;
*/

};

/*
* 3. 기타 I2C, SPI 등 ANTSDR E300의 다른 하드웨어 설정
* (예: &i2c0, &spi0 등에 status = "okay" 및 하위 장치 정의)
*/

```

D. (표) ANTSDR E300 디바이스 트리 노드 매핑

디바이스 트리는 하드웨어(HW)와 소프트웨어(SW)를 연결하는 "지도"입니다. 사용자는 `realtek,rtl8211cl`이라는 문자열이 왜 커널의 `realtek.c` 드라이버를 로드하는지, 그리고 `phy@1`의 의미가 무엇인지 직관적으로 알기 어렵습니다. 다음 표는 ANTSDR E300의 주요 칩(HW)과 Vivado 설정, 그리고 dtsi 파일의 노드(SW) 간의 인과관계를 명확하게 매핑합니다.

하드웨어 (칩)	스키매틱/Vivado 설정	DTSI 노드 (system-user.dtsi)	커널 드라이버 (.cfg)
Realtek RTL8211CL	GEM0, MIO 50/51	&gem0, phy@1, compatible = "realtek,rtl8211cl"	CONFIG_REALTEK_PHY
Analog AD9361	AXI IP @ 0x43C00000, IRQ_F2P	ad9361@43C00000, compatible = "adi,ad9361-phy"	CONFIG_AD9361
SD Card	SDIO 0 (MIO 40-45)	&sdhci0	(기본 활성화됨)

UART	UART 1 (MIO 48-49)	&uart1	(기본 활성화됨)
------	--------------------	--------	-----------

V 단계(커널 설정)와 VI 단계(DTSI 작성)는 PYNQ 포팅의 심장부이며, 이 두 가지가 완벽하게 일치하지 않으면 S_ERR_O2와 같은 치명적인 부팅 실패가 발생합니다.

이 둘의 인과관계는 다음과 같습니다:

1. 사용자는 CONFIG_REALTEK_PHY=y (V 단계)를 설정하여 Linux 커널에 realtek.c 드라이버를 포함시켰습니다.
2. 또한 system-user.dtsi (VI 단계)에 compatible = "realtek,rtl8211cl" 노드를 추가했습니다.
3. 부팅 시, 커널은 system-user.dtsi를 읽고 이 compatible 문자열을 확인합니다.
4. 커널은 내장된 드라이버 목록(V 단계에서 생성됨)을 탐색하여 "realtek,rtl8211cl" 문자열을 처리할 수 있는 드라이버(realtek.c)를 찾습니다.
5. 드라이버를 찾으면 해당 노드(phy@1)의 reg = 정보를 사용하여 MDIO 버스의 1번 주소에 있는 PHY 칩과 통신을 시도(probe)합니다.
 - 실패 시나리오 1: V 단계가 누락되면 (커널에 드라이버가 없음), 커널은 compatible 문자열에 맞는 드라이버를 찾지 못하고 eth0: PHY driver not found 메시지를 출력합니다.
 - 실패 시나리오 2: VI 단계가 누락되면 (DTSI에 노드가 없음), V 단계에서 드라이버를 빌드했더라도 커널이 어떤 하드웨어에 이 드라이버를 적용해야 할지 모르므로 아무 작업도 하지 않습니다 (이더넷이 동작하지 않음).

결론적으로, V 단계와 VI 단계는 논리적인 AND 조건이며, 포팅 성공을 위해 두 단계 모두 완벽해야 합니다.

VII. 4단계: PYNQ 3.1.1 이미지 빌드 및 통합

이제 모든 준비가 완료되었습니다. 1단계에서 생성한 하드웨어 정의(XSA)와 2, 3단계에서 준비한 PYNQ 보드 패키지(spec, cfg, dtsi 파일)를 사용하여 sdbuild를 실행합니다.

A. sdbuild 실행

1. PYNQ 3.1.1용으로 사전 빌드된 ARMv7 (Zynq) 루트 파일 시스템(RootFS) 이미지를 PYNQ 웹사이트에서 미리 다운로드해야 합니다. (예: pynq_rootfs_arm_v3.1.1.img.gz)
2. 압축을 해제하여 pynq_rootfs_arm_v3.1.1.img 파일을 준비합니다.
3. 모든 설정 파일(XSA, .spec, .cfg, .dtsi)이 PYNQ/boards/ANTSDR_E300/ 디렉토리 하위 디렉토리에 올바르게 위치했는지 최종 확인합니다.

4. PYNQ 소스 코드의 `sdbuild` 디렉토리로 이동합니다.

```
Bash
cd PYNQ/sdbuild
```

5. Xilinx 도구 환경 변수를 현재 터미널에 적용합니다.

```
Bash
# Petalinux 2023.1 및 Vivado 2023.1 환경 변수 스크립트를 먼저 source 해야 함
source /opt/Xilinx/Vivado/2023.1/settings64.sh
source /opt/Xilinx/Petalinux/2023.1/settings.sh
```

6. `make` 명령어를 실행하여 ANTSDR E300 보드에 대한 이미지 빌드를 시작합니다.

```
Bash
# ANTSDR_E300 보드에 대한 이미지 빌드 실행
# PREBUILT 경로는 다운로드한 RootFS 이미지의 절대 경로로 지정
make BOARD=ANTSDR_E300 PREBUILT=/path/to/pynq_rootfs_arm_v3.1.1.img
```

B. 빌드 프로세스 분석

`make` 명령어는 IV-B 단계에서 작성한 `ANTSDR_E300.spec` 파일을 읽고 다음과 같은 복잡한 과정을 자동으로 수행합니다.

1. `zynq.bsp`를 기반으로 임시 `Petalinux` 프로젝트를 생성합니다.
2. `ANTSDR_E300.xsa`를 `Petalinux` 프로젝트에 적용하여 하드웨어 구성을 업데이트합니다.
3. `PETALINUX_CONFIG_ANTSDR_E300` 디렉토리에 있던 `bsp.cfg`와 `system-user.dtsi`를 `Petalinux` 프로젝트의 `meta-user` 레이어에 복사합니다.
4. `petalinux-build`를 실행합니다. 이 과정에서 3단계에서 수정한 설정을 바탕으로 U-Boot, Linux 커널, 디바이스 트리(.dtb)가 컴파일됩니다.
5. 빌드가 완료되면, `PREBUILT`로 지정된 `pynq_rootfs_arm_v3.1.1.img` 이미지를 마운트합니다.
6. `Petalinux` 빌드 결과물(부트로더 `BOOT.BIN`, 커널 `image.ub` 등)을 마운트된 이미지의 부팅 파티션(FAT32)에 복사(주입)합니다.
7. 모든 작업이 완료되면, 최종적으로 `ANTSDR_E300-v3.1.1.img` 파일을 `PYNQ/sdbuild/output/` 디렉토리에 생성합니다.

VIII. 배포 및 시스템 검증

A. SD 카드 이미지 플래시

1. PYNQ/sdbuild/output/ANTSDR_E300-v3.1.1.img 파일을 찾습니다.
2. Balena Etcher, Rufus 또는 dd와 같은 이미지 라이팅 도구를 사용하여 SD 카드에 이미지를 플래시합니다.

Bash

(주의!) 'of=/dev/sdX'의 X는 SD 카드 장치명입니다.

대상 장치를 잘못 지정할 경우 호스트 시스템의 데이터가 영구적으로 손상될 수 있습니다.

lsblk 명령으로 정확한 장치명을 확인하십시오.

```
sudo dd if=output/ANTSDR_E300-v3.1.1.img of=/dev/sdX bs=4M status=progress  
conv=fsync
```

B. 첫 부팅(First Boot) 및 UART 모니터링

1. ANTSDR E300 보드에 플래시된 SD 카드를 삽입합니다.
2. 보드의 UART 포트(III-C 단계에서 MIO 48/49로 설정한)를 PC의 USB 포트에 연결합니다.
3. minicom, putty 또는 screen과 같은 시리얼 터미널 에뮬레이터를 엽니다. (Baudrate: 115200)
4. 보드의 전원을 켭니다.

C. 부팅 로그 검증 (성공의 증거)

터미널에 U-Boot 및 Linux 커널 부팅 로그가 출력됩니다. 단순한 부팅이 아닌, V/VI 단계에서 의도한 하드웨어 드라이버가 올바르게 로드되는지 확인하는 것이 중요합니다.

다음 표는 성공적인 포팅을 검증하기 위한 핵심 로그 메시지 체크리스트입니다.

부팅 단계	확인해야 할 핵심 로그 메시지 (예시)	실패 시 (S_ERR_02)
U-Boot	Hit any key to stop autoboot: 0	(U-Boot 로드 실패)

Kernel	Linux version 6.1.0-xilinx-v2023.1...	(커널 로드 실패)
Ethernet PHY	libphy: mdio (s) : &gem0... eth0: PHY (driver?) attached to... eth0: Link is Up - 1Gbps/Full - flow control rx/tx	eth0: PHY [realtek-rtl8211cl] driver not found (V 또는 VI 단계 오류)
SDR (IIO)	ad9361 spi0.0: ad9361_probe... AD9361 Rev 2 successfully initialized	ad9361: probe of... failed with error -2 (ENODEV) (V 또는 VI 단계 오류)
PYNQ Service	[OK] Started Jupyter Notebook server.	(RootFS 마운트 실패)
RootFS Mount	EXT4-fs (mmcblkOp2): mounted filesystem...	Kernel panic - not syncing: VFS: Unable to mount root fs (SD 카드 드라이버 오류, X 부록 참조)

D. 네트워크 및 PYNQ 서비스 검증

부팅이 완료되고 PYNQ 로그인 프롬프트가 나타나면, 기본 계정(ID: xilinx, 암호: xilinx)으로 로그인합니다.

1. **네트워크 검증:** `ifconfig eth0` 또는 `ip a` 명령을 실행하여 `eth0` 인터페이스가 이더넷 드라이버에 의해 인식되고, DHCP를 통해 IP 주소를 할당받았는지 확인합니다.
2. **SSH 접속:** 호스트 PC에서 `ssh xilinx@<ip_address>`를 실행하여 원격 접속이 가능한지 확인합니다.
3. **Jupyter 검증:** 호스트 PC의 웹 브라우저에서 `http://<ip_address>:9090` (또는 `http://antsdr-e300.local:9090`)으로 접속하여 PYNQ Jupyter Notebook 인터페이스가

로드되는지 확인합니다. (암호: xilinx)

IX. 결론: 포팅 완료 및 후속 조치

A. 포팅 성공 요약

이 가이드의 절차를 성공적으로 따랐다면, ANTSDR E300 하드웨어(XC7Z030, RTL8211CL, AD9361)에 최적화된 PYNQ 3.1.1 부팅 이미지를 성공적으로 빌드 및 배포한 것입니다. 시스템은 이제 PYNQ 프레임워크를 통해 Python으로 하드웨어(PL) 및 SDR 프론트엔드(AD9361)를 제어할 준비가 되었습니다.

B. 후속 조치 (SDR 라이브러리 설치)

PYNQ 환경 내에서 AD9361을 제어하려면, Analog Devices의 libiio 라이브러리 및 Python 바인딩(pyadi-iio)을 설치해야 합니다.

부팅된 ANTSDR E300의 터미널(SSH 또는 UART)에서 다음 명령어를 실행하여 pyadi-iio 패키지를 설치합니다. (이더넷 연결이 필요합니다.)

```
Bash
```

```
pip install pyadi-iio
```

설치가 완료되면, Jupyter Notebook을 열고 새 노트북에서 `import adi`를 실행하여 AD9361 RF Transceiver를 Python 객체로 제어할 수 있습니다.

X. 부록: 주요 문제 해결(Troubleshooting) 가이드

포팅 과정은 복잡하며 여러 지점에서 오류가 발생할 수 있습니다. 다음은 일반적인 오류 사례와 해결 방법입니다.

사례 1: Vivado XSA 생성 실패 (S_ERR_01 유사 오류)

- 증상: make 실행 시 Petalinux 빌드 초기에 ERROR: [Common 17-55] 'get_property' expects at least one object.와 같은 오류가 발생하며 중단됩니다.
- 원인: 1단계(Vivado)에서 생성된 ANTSDR_E300.xsa 파일이 불완전합니다. Zynq PS의 DDR3 설정이 누락되었거나, "Validate Design"을 통과하지 못한 디자인을 기반으로 XSA가 생성되었을 수 있습니다.
- 해결: III-C 단계를 다시 검토하여 DDR, MIO, Ethernet 핀 설정이 ANTSDR E300 스키매틱과 정확히 일치하는지 확인합니다. "Validate Design"을 다시 실행하고, Bitstream을 포함하여 XSA를 재생성한 후 4단계(make)를 다시 시도합니다.

사례 2: Petalinux 빌드 오류 (커널 드라이버 컴파일 실패)

- 증상: make 실행 중 ERROR: linux-xlnx-6.1-r0 do_compile: ExecutionError(...) 로그가 발생하며, 세부 로그에 CONFIG_REALTEK_PHY 관련 심볼(symbol)을 찾을 수 없다는 내용이 표시됩니다.
- 원인: V-A 단계에서 생성한 bsp.cfg 파일에 오타가 있거나, 파일 경로가 PYNQ/boards/ANTSDR_E300/petalinux_config/project-spec/meta-user/recipes-kernel/linux/files/bsp.cfg가 아니어서 Petalinux 빌드 시스템이 해당 설정 조각을 인식하지 못했습니다.
- 해결: bsp.cfg 파일의 경로와 내용을 다시 확인합니다.

사례 3: 부팅 실패 (S_ERR_02: Kernel panic - not syncing: VFS: Unable to mount root fs)

- 증상: 커널 부팅 로그 마지막에 루트 파일 시스템(SD 카드의 두 번째 파티션)을 마운트하지 못하고 패닉 상태가 됩니다.
- 원인: 이 오류는 증상일 뿐, 근본 원인은 SD 카드 컨트롤러 드라이버 로드 실패입니다.
- 해결 (논리적 추론):
 1. &sdhci0 (SD 카드 컨트롤러)가 system-user.dtsi에서 올바르게 활성화(status =

- "okay") 되었는지 확인합니다. (PYNQ 기본 BSP는 보통 활성화합니다.)
2. 가장 가능성이 높은 원인: 1단계(Vivado) III-C에서 "SDIO 0"가 올바른 MIO 핀(40-45)에 할당되었는지 XSA를 재확인합니다.
 3. 이 하드웨어 정의(XSA)가 잘못되면 Petalinux가 생성한 sdhci0 노드 자체가 잘못되어 커널이 SD 카드를 인식하지 못합니다.
 4. 커널이 SD 카드를 인식하지 못하면, SD 카드의 두 번째 파티션(RootFS)을 마운트할 수 없어 VFS(Virtual File System) 패닉이 발생합니다.
 5. Vivado MIO 설정을 수정한 후 XSA를 재생성하고, 4단계(make)부터 다시 진행해야 합니다.

사례 4: 부팅 실패 (S_ERR_02: eth0: PHY [realtek-rtl8211cl] driver not found)

- 증상: 이더넷이 동작하지 않고, 부팅 로그(dmesg)에 PHY 드라이버를 찾을 수 없다고 나옵니다.
- 원인: VI-E에서 설명한 V 단계와 VI 단계의 불일치.
- 해결:
 1. (커널 설정 확인): V-A 단계의 bsp.cfg에 CONFIG_REALTEK_PHY=y가 포함되어 있는지 확인합니다. (드라이버가 커널에 존재하는가?)
 2. (디바이스 트리 확인): VI-A 단계의 system-user.dtsi에 compatible = "realtek,rtl8211cl"이 정확히 포함되어 있는지 확인합니다. (커널에게 드라이버를 사용하라고 지시했는가?)
 3. (하드웨어 정의 확인): III-C 단계에서 Vivado의 Zynq PS 설정에 GEMO가 올바른 MIO 핀(50/51)으로 활성화되었는지 확인합니다. (MDIO 버스가 존재하는가?)
 4. 세 가지 중 하나라도 누락되면 이더넷은 동작하지 않습니다.